

TETRIS++

By *Velvetal*

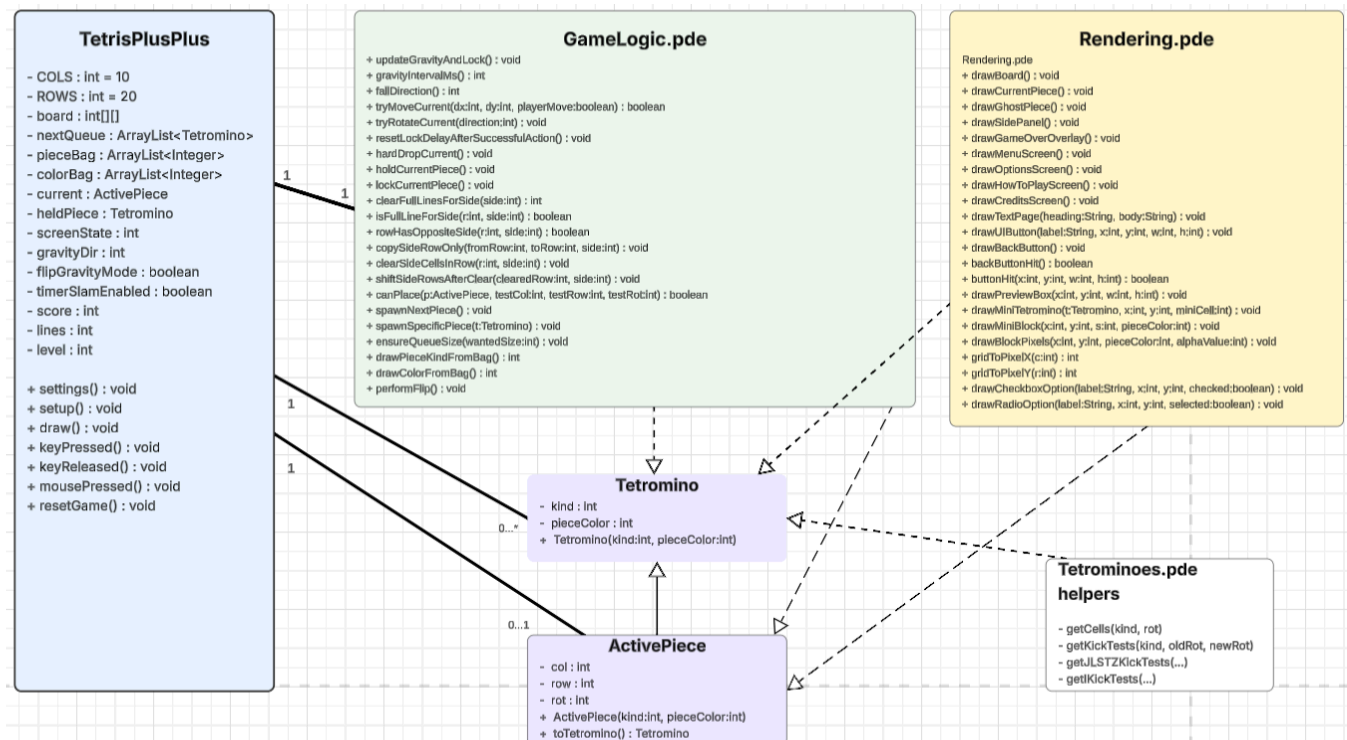
Hudson Dong, Period 6

DESCRIPTION:

Tetris++ is a spin-off of the traditional block stacking game of Tetris! Specifically, I introduce a new type of move: the flip. Whether I reverse the board or revert gravity, this twist is sure to bring new life into the game! I retain the most important functionalities of gameplay which includes proper board rendering, 7-bag piece and color randomization, multi-row clearing and point accumulation, piece holding and spawning, soft and hard drops, speed and timing, the ghost piece, and gameplay resets. Additionally, I will be implementing both board flips and gravity flips as a switch at a key press. Consequently I will also adjust spawning and game-over detection mechanics accordingly.

In terms of libraries, the project of course uses the built-in Processing core library for graphics, input, and the game window. Processing functions such as `size()`, `draw()`, `rect()`, `fill()`, `text()`, `keyPressed()`, `keyReleased()`, and `mousePressed()` are used to draw the board, pieces, buttons, text, and handle user input. The project also uses standard Java features such as arrays, `ArrayList`, and `Collections.shuffle()` to store the board, manage pieces, and randomize the 7-bag system.

UML DIAGRAM (LUCIDCHART):



HOW DOES IT WORK?

To start the project, the user, obviously, should open and run the main .pde file.

Upon startup, the user should see the main menu screen titled Tetris++. From there, the user can click "Play" to begin the game, "Options" to toggle the various built-in settings, "How to Play" to read the instructions, or "Credits" to view the credits page.

Once the game begins, tetromino pieces automatically start falling from the top of the board. The player controls each falling piece and tries to place it so that full horizontal lines are created. When a row is completely filled with blocks, that row disappears, the player earns points, and the blocks above it move downward. The game continues until the blocks stack enough so that a new piece has no more space to spawn in.

The controls are:

- Left Arrow - Move the piece left
- Right Arrow - Move the piece right
- Down Arrow - "Soft drop," which speeds up the falling of the current piece
- Up Arrow - Hold or swap into held piece slot
- A - Rotate the piece counterclockwise
- S - Hard drop, which instantly drops and locks the current piece
- D - Rotate the piece clockwise
- R - Restart the game
- F - Flip gravity
- ESC - Return to the menu

The game works using Processing's draw loop. During each frame, the program checks which screen the player is currently on. If the player is on the "Menu," "Options," "How to Play," or "Credits" screen, the program draws that screen and waits for the player to interact with a button. If the player is on the game screen, the program updates the falling piece, applies gravity, checks keyboard input, handles movement and rotation, locks pieces in the place if applicable, clears completed lines, updates the score, and redraws the board repeatedly. The board is stored as a grid with 10 columns and 20 rows. Empty spaces are stored as blank cells, while filled spaces store the color of a locked block. The current falling piece is separate from the locked board until it reaches the bottom or lands on another block. After the piece locks into place, the program checks for full rows. If any full rows are found, they are cleared, the score is updated, and a new piece spawns from the next queue. The game also uses a 7-bag randomizer to choose pieces. This means all seven tetromino pieces are placed into a shuffled bag, and pieces are taken from that bag one at a time. Once the bag is empty, a new shuffled bag is created. This makes the random piece order more balanced because each group of seven pieces contains one of each tetromino. The game also includes a hold feature, a next-piece preview, and a ghost piece option. The hold feature lets the player save a piece for later or swap it with the current piece. The next-piece preview shows the next two pieces that will appear. The ghost piece shows a transparent preview of where the current piece will land, and it can be turned on or off from the Options screen. Finally, the game currently supports a "flip gravity" feature, which does exactly what it says: gravity enters a reversed mode in which the piece falls up into a new pile at the top of the board. Normal line clearing rules apply here, so gameplay continues, just on both ends of the board. Spawns are now on the opposite side of the current gravitational direction. The game is adjusted accordingly to end when there is no spawning space instead of occupied top rows, and score incrementation is operational for all full rows or soft or hard drops to the top.

Current Functionalities	Planned Functionalities
- settings() - setup() - draw() - keyPressed() - keyReleased() - mousePressed() - resetGame() - Tetromino(int kind, int pieceColor) - ActivePiece(int kind, int pieceColor) - toTetromino() - getCells(int kind, int rot) - getKickTests(int kind, int oldRot, int newRot) - getJLSTZKickTests(int oldRot, int newRot) - getIKickTests(int oldRot, int newRot) - updateGravityAndLock() - gravityIntervalMs() - fallDirection() - tryMoveCurrent(int dx, int dy, boolean playerMove) - tryRotateCurrent(int direction) - resetLockDelayAfterSuccessfulAction() - hardDropCurrent() - holdCurrentPiece() - lockCurrentPiece() - clearFullLinesForSide(int side) - isFullLineForSide(int r, int side) - rowHasOppositeSide(int r, int side) - copySideRowOnly(int fromRow, int toRow, int side) - clearSideCellsInRow(int r, int side) - shiftSideRowsAfterClear(int clearedRow, int side) - canPlace(ActivePiece p, int testCol, int testRow, int testRot) - spawnNextPiece()	- flipBoardRows() - performFlip() - currentPieceStillValidAfterFlip() - handleInvalidPieceAfterBoardFlip() - keyPressed() - mousePressed() - drawOptionsScreen() - drawHowToPlayScreen() - fallDirection() - updateGravityAndLock() - clearFullLinesForSide(int side) - shiftSideRowsAfterClear(int learedRow, int side) - canPlace(ActivePiece p, int testCol, int testRow, int testRot) - + additional helpers/middle-man methods Updates on: - resetGame() - updateGravityAndLock() - gravityIntervalMs() - tryMoveCurrent() - tryRotateCurrent() - hardDropCurrent() - clearFullLines() - spawnNextPiece() - spawnSpecificPiece() - drawGhostPiece() - drawOptionsScreen()

- spawnSpecificPiece(Tetromino t)
- ensureQueueSize(int wantedSize)
- drawPieceKindFromBag()
- drawColorFromBag()
- performFlip()
- drawBoard()
- drawCurrentPiece()
- drawGhostPiece()
- drawSidePanel()
- drawGameOverOverlay()
- drawMenuScreen()
- drawOptionsScreen()
- drawHowToPlayScreen()
- drawCreditsScreen()
- drawTextPage(String heading, String body)
- drawUIButton(String label, int x, int y, int w, int h)
- drawBackButton()
- backButtonHit()
- buttonHit(int x, int y, int w, int h)
- drawPreviewBox(int x, int y, int w, int h)
- drawMiniTetromino(Tetromino t, int x, int y, int miniCell)
- drawMiniBlock(int x, int y, int s, int pieceColor)
- drawBlockPixels(int x, int y, int pieceColor, int alphaValue)
- gridToPixelX(int c)
- gridToPixelY(int r)
- drawCheckboxOption(String label, int x, int y, boolean checked)
- drawRadioOption(String label, int x, int y, boolean selected)